

Kurs-Agent — Anforderungen

Stand 05.10.2026. 96 Punkte in elf Gruppen. Jede Anforderung ist so gefasst, dass sie **scheitern kann**: es steht dabei, woran man sie misst. Was nur maschinell prüfbar ist, prüft `pruefe_alles.py`.

Die Aufgabe — gesetzt am 04.10.2026, 12:40 Uhr (Quelle: AUFGABE.md): „Ziel der Kurs-KI ist es, auf einem Ordner ein SLM oder kleines LLM speziell für die Python-Programmierung zusammen mit einem dafür passenden Agenten und den Python-Werkzeugen auf einen einzigen Ordner zu installieren. Das System soll Python-Programme auf Anforderung schreiben und selbst testen können, um Syntaxfehler über den Agenten selbst zu korrigieren, dann entweder simulieren oder direkt auf die Hardware laden. Das alles muss so geschehen, dass der Anwender nichts laden muss; es muss ohne Internetzugang funktionieren, auch alle Treiber müssen vorhanden sein. Es wird nichts installiert außer eventuell nötigen Treibern, und alles läuft nur mit den Werkzeugen auf dem besagten Ordner; es wird kein schon vorhandenes Python verwendet, keine Libs, die irgendwo gefunden wurden, nichts sonst — alles muss mitgeliefert werden. Ich will nicht sehen: ‚es muss noch nachgeladen werden‘.“

Der Maßstab über allem — gesetzt am 04.10.2026: „Alles muss echt laufen auf meinem Rechner, alles inklusive LLM, Agent, Werkzeugen, Hardware. Das ist gesetzt.“ Keine Anforderung dieses Blatts gilt als erfüllt, solange sie nur unter Wine, im Simulator, im Browser-Nachbau oder auf dem Linux-Prüfstand bestanden hat. Erfüllt heißt: **auf C:\Kurs_Agent, mit Protokoll und Bildschirmfoto.** Den Stand je Säule führt `PRUEFPROTOKOLL.md` an erster Stelle; am 04.10.2026 fehlt darin noch das Sprachmodell (Smart App Control sperrt `llama.cpp` auf diesem Rechner).

A — Autarkie

Der Kursrechner hat **Windows und sonst nichts**. Kein Python, kein Netz, keine Adminrechte.

	Anforderung	geprüft durch
A1	Das Paket bringt alles mit, was es braucht: Python, pip, Pakete, Firmware, Sprachmodell, Treiber.	<code>agent/pfade.py</code> meldet jeden Teil als „gefunden“
A2	Im Kurs wird nichts aus dem Netz geholt.	Lauf mit getrennter Verbindung
A3	Es wird nichts installiert: kein Installationsprogramm, keine Registrierung, kein Dienst, keine Adminrechte.	Durchsicht aller Programmtexte; <code>nachweis_autark</code>
A4	Im Kursbetrieb wird ein vorhandenes Python nicht gesucht und nicht benutzt. Jeder Aufruf geht über das Python aus dem Paket.	<code>pruefe_alles.py</code> A4: keine Suche außerhalb des Füllzweigs, das gefundene Python nur für <code>hole_paket.py</code> , <code>python_exe()</code> zeigt stets in die Ablage — mit Gegenprobe
A5	Kein fester Pfad und kein fester Laufwerksbuchstabe. Alles leitet sich vom Ort der <code>START.bat</code> ab.	Suche nach <code>C:\</code> in allen Programmtexten

	Anforderung	geprüft durch
A6	Ohne Sprachmodell läuft alles Übrige weiter — Werkzeuge, Abnahme, virtuelle LED, Flashen, Nachweis, Aufräumen.	agent.py --drehbuch
A7	Doppelklick genügt auch beim ersten Mal. Ist das Paket leer oder unvollständig, sagt START.bat das, sucht für diesen einen Schritt ein Python mit pip — jeden Fundort, nicht nur den ersten; der Platzhalter des Windows-Stores wird am Namen übersprungen —, nennt es und füllt. Keine Kommandozeile, kein getippter Befehl.	pruefe_alles.py F8; Lauf unter Wine; erster echter Windows-Lauf 03./04.10.2026
A8	Vollständig heißt alle Teile. Ob zu füllen ist, entscheidet nicht eine Datei (Python), sondern die Liste aller Bestandteile in agent/pfade.py (--fehlt). Ein halb gefülltes Paket geht nie stillschweigend ins Menü.	pfade.py --fehlt liefert 1, solange etwas fehlt; START.bat fragt danach vor jedem Start
A9	Der Vorrat kommt vor dem Netz — für jede Datei. Was schon auf dem Rechner liegt (Paket nebenan, X:\Kurs_Agent\paket auf jedem Laufwerk, C:\Projekte\Kurs_Agent\paket, Downloads), wird kopiert, nicht geladen; ohne Netz wird gar kein Download versucht. Liegt irgendwo ein volles Paket, füllt sich ein neues ohne Internet.	Gegenprobe in hole_paket.py (Vorrat ohne Netz → kopiert; nichts im Vorrat ohne Netz → klare Abweisung)
A10	Laden heißt fertig laden. Eine Datei gilt erst als da, wenn sie vollständig ist (.teil bis zum Schluss); ein Abbruch wird an derselben Stelle fortgesetzt (HTTP Range), drei Anläufe je Datei, und ein gescheiterter Schritt reißt die übrigen nicht mit.	laden(); Füllprotokoll

	Anforderung	geprüft durch
A11	Eine Signaturpflicht des Rechners (Smart App Control) wird vor dem Start erkannt und in Worten gemeldet — nicht erst durch ein Fehlerfenster. Sperrt sie llama.cpp, läuft der Agent von selbst ohne Modell weiter (A6), mit demselben Ablauf und derselben Abnahme; Grund und Ausweg stehen in der Oberfläche.	<code>modell.signaturpflicht();</code> <code>pruefe_rechner.py --kurz;</code> Ereignis <code>kein_modell</code> in der Oberfläche
A12	Jeder Lauf hinterlässt eine Spur. <code>START.bat</code> schreibt <code>letzter_lauf.txt</code> mit Marke an jeder Weggabelung, <code>hole_paket.py</code> schreibt <code>fuellen_protokoll.txt</code> samt vollständigem Fehlerbericht. Ein Lauf, der keine Spur hinterlässt, lässt sich nicht beurteilen.	beide Dateien nach jedem Lauf im Paketordner

Was der erste echte Windows-Lauf lehrte (03./04.10.2026): Unter Wine war all das geprüft und grün. Auf dem echten Rechner scheiterte der erste Start an vier Dingen, die Wine nicht kennt — dem Store-Platzhalter von `where python`, einer Zeitüberschreitung bei GitHub, dem „fertig“-Kriterium am halb gefüllten Paket und der Signaturpflicht von Smart App Control. Keines davon war ein Fehler des Konzepts; jedes war ein Fehler darin, was als geprüft galt. Daraus A8–A12.

Ausnahme, benannt und eng gefasst: Das **Füllen** des leeren Pakets braucht einmalig ein Python mit pip und eine Internetverbindung — im leeren Paket liegt ja noch keines. Diese eine Stelle sucht eines, prüft ob pip darin läuft, nennt es beim Namen und fragt, bevor sie etwas holt. Sie benutzt es **ausschließlich** für `hole_paket.py`; jeder andere Aufruf im ganzen Vorgang geht über das Python aus dem Paket. Auch dabei bleibt der Rechner unberührt: `PIP_CACHE_DIR` und `PYTHONNOUSERSITE` zeigen in den Kursordner, nicht ins Benutzerprofil (B3, B4). Das geschieht auf dem Vorbereitungsrechner, nie auf dem Kursrechner — dorthin wandert der **gefüllte** Ordner.

Nicht Teil des Pakets: Die Prüfbrücke, über die dieses Paket während der Erprobung aus der Ferne beobachtet wird, gehört nicht dazu. Das Paket kennt sie nicht, braucht sie nicht und läuft ohne sie; nachgeprüft: kein Programmtext des Pakets nennt sie.

B — Keine Störung dessen, was schon da ist

Läuft auf dem Rechner bereits Thonny, eine Arduino-Umgebung oder ein eigenes Python, darf der Kurs daran nichts ändern und nichts blockieren.

	Anforderung	geprüft durch
B1	Ein vorhandenes Python wird nicht gestartet, nicht gelesen, nicht verändert.	A4

	Anforderung	geprüft durch
B2	Dessen site-packages bleiben unberührt; es wird nie dorthin installiert.	nachweis_autark vorher/nachher
B3	Das Benutzer-Paketverzeichnis (%APPDATA%\Python) wird nicht beschrieben.	PYTHONNOUSERSITE=1 in umgebung()
B4	Der pip-Zwischenspeicher des Benutzers wird nicht benutzt.	PIP_CACHE_DIR zeigt in die Ablage
B5	PATH und die Umgebungsvariablen des Rechners bleiben unverändert.	setlocal in START.bat; nachweis_autark
B6	Ein belegter Netzwerkport wird nicht übernommen, sondern ausgewichen.	Gegenprobe: Port besetzen, Oberfläche starten
B7	Ein belegter COM-Anschluss wird nicht erzwungen; es wird gemeldet, wer ihn hält.	Fehlermeldung von esp32_firmware
B8	Zwei gleichzeitige Läufe im selben Ordner werden verhindert, nicht geduldet.	Sperrdatei; Gegenprobe
B10	Jeder Programmstart reicht die abgeschirmte Umgebung weiter — ein einziger ohne sie genügt für ein halb fremdes Python.	pruefe_alles.py B10, mit Gegenprobe

C — Nur im Kursordner schreiben

	Anforderung	geprüft durch
C1	Geschrieben wird ausschließlich in <Kursordner>\ablage\.	in_der_ablage() im Werkzeugrahmen
C2	Ein Pfad, der aus der Ablage hinausführt, wird abgewiesen — auch über ...	Gegenproben mit ../../tmp, /etc, C:\Windows
C3	Absolute Pfade werden abgewiesen, nicht stillschweigend umgedeutet.	Gegenprobe
C4	Während eines vollständigen Laufs entsteht außerhalb des Kursordners keine Datei und ändert sich keine.	beweis.py: Dateisystem vorher/nachher
C5	Das mitgelieferte paket\ wird nur gelesen, nie beschrieben.	Prüfsumme vorher/nachher

Ausnahme, benannt: Öffnet die Oberfläche den Browser, schreibt **der Browser** in sein eigenes Profil (Verlauf, Zwischenspeicher). Das ist eine Spur außerhalb des Kursordners, die nicht vom Paket stammt, aber von ihm ausgelöst wird. Wer das nicht will, ruft die Adresse von Hand auf.

Zweite Ausnahme, benannt: Der Wächter lässt %TEMP% zu (waechter._erlaubt). Nicht aus Nachlässigkeit, sondern weil Python selbst dort arbeitet — beim Entpacken, beim Übersetzen, bei jedem tempfile. Ein Schreibversuch dorthin wird also **nicht** gemeldet. Windows räumt diesen Ordner selbst auf; eine bleibende Veränderung am Rechner entsteht nicht. Wer es genau wissen will: pruefe_unversehrtheit.py zählt auch die Spuren in %TEMP% mit und nennt sie einzeln.

D — Restlos entfernbar

	Anforderung	geprüft durch
D1	Alles Erzeugte liegt in einem Ordner und wird mit einem Befehl gelöscht.	aufraeumen
D2	Nach dem Löschen wird nachgezählt; Reste werden mit Grund gemeldet.	aufraeumen zählt nach
D3	Ein Programm, das eine Datei offen hält (der Modellserver), wird vorher beendet — und nur dieses.	Prozessnummer aus llama.pid, Gegenprobe
D4	Nach dem Aufräumen ist der Rechner nachweislich im Ausgangszustand.	nachweis_autark vorher/nachher
D5	Derselbe Ordner läuft danach wieder, beliebig oft.	drei volle Zyklen, Prüfsumme gleich
D6	Bei Betrieb aus der Kopie löscht sich der Arbeitsordner selbst.	START.bat, Betriebsart [2]

E — Weitergabe

	Anforderung	geprüft durch
E1	Das Paket läuft vom USB-Stick und von der Festplatte, ohne Änderung.	A5
E2	Ein Befehl stellt den weitergabefertigen Ordner zusammen, ohne Entwicklungsreste.	baue_stick.py
E3	Fehlt etwas im Paket, wird es beim Bauen und beim Start gemeldet, nicht erst im Kurs.	pfade.bericht(), baue_stick.py
E4	Derselbe Stick startet auf Rechnern verschiedener Größe; das Modell wird beim Start gewählt.	modell.waehlen(), Gegenproben bei 52/7,5/5/3/1 GB
E5	Der Vorrat ist vollständig: jedes erlaubte Paket liegt bei, und jede geltende Abhängigkeit davon.	pruefe_alles.py E5 rechnet die Metadaten durch; Gegenprobe

F — Bedienung

	Anforderung	geprüft durch
F1	Doppelklick auf <code>START.bat</code> genügt; keine Kommandozeile nötig.	—
F2	Die Aufgaben sind anklickbar, nicht zu tippen.	Oberfläche, vier Karten
F3	Jeder Schritt ist sichtbar: Werkzeug, Werte, Ergebnis, Urteil.	Oberfläche
F4	Ein gesperrter Knopf sagt, warum.	Oberfläche sperrt während des Laufs, Ampel zeigt „läuft“
F5	Jede Fehlermeldung nennt die Ursache und den nächsten Schritt.	Durchsicht aller Meldungen
F6	Löschen wird bestätigt, bevor es geschieht.	Rückfrage im Browser
F7	Die Anleitung nennt dieselben Menünummern wie <code>START.bat</code> — wer ihr folgt, drückt die richtige Taste.	<code>pruefe_alles.py</code> F7, mit Gegenprobe
F8	Die Kurs-Seite (Anleitung) enthält eine genaue Beschreibung mit Funktionsskizze : was das Sprachmodell tut, was der Agent tut (Schleife, Regeln, Abnahme, Gedächtnis des Gesprächs), was jedes Werkzeug tut, wie Erwartung, Prüfung und Gerät zusammenhängen — als beschriftete Zeichnung, nicht als Aufzählung. Anweisung vom 04.10.2026: „im Kurs-HTML auch später die genaue Beschreibung mit schöner Funktionsskizze hinterlegen: Beschreibung des LLM, des Agenten, der Werkzeuge etc.“ Offen.	Durchsicht der Anleitung; die Skizze wird aus den Werkzeugbeschreibungen erzeugt, nicht abgeschrieben

G — Der Lehrinhalt

Das Paket zeigt denselben Entwicklungsprozess, mit dem auch ernsthafte Arbeit entsteht.

	Anforderung	geprüft durch
G1	Das Modell schreibt das Programm, prüft es selbst und liefert erst dann aus.	Anweisung, Drehbuch, echter Lauf
G2	Der Prüfschritt ist eine Abnahme gegen eine vorher genannte Erwartung , keine Vorführung.	erwartet-Feld; ohne Erwartung sagt das Werkzeug das
G3	Die Abnahme kann durchfallen — und tut es bei falschem Takt, falschem Anschluss, fehlendem Takt, Absturz.	vier Gegenproben
G4	Ein Agent darf sich nicht für fertig erklären, solange die letzte Prüfung fehlschlug.	pruefe_schleife.py, acht Punkte
G5	Ein Fehler ist eine Nachricht, kein Abbruch: das Modell bekommt ihn und berichtet.	echter Lauf: NICHT BESTANDEN → berichtet → BESTANDEN
G6	Dieselbe Datei, die geprüft wurde, geht unverändert auf die Hardware.	programm_testen ändert den Quelltext nicht
G7	Die Erwartung setzt der Mensch : steht sie im Auftrag (pins, takt_hz), gilt genau sie. Das Modell kann sie nicht umschreiben, damit sein Programm besteht.	pruefstand/pruefe_regeln.py: Modell nennt 0,5 Hz, Auftrag 1,0 Hz → geprüft wird 1,0 Hz, Hinweis an das Modell
G8	Eine Änderung, die seit der letzten Prüfung nicht geprüft wurde, oder eine durchgefallene letzte Prüfung lässt kein FERTIG zu. Das Modell wird bis zu dreimal zurückgewiesen; dann endet der Lauf als NICHT ABGENOMMEN , nie als FERTIG.	pruefstand/pruefe_ungeprueft.py: ungeprüfte Fassung → abgewiesen → geprüft → FERTIG; dreimal FERTIG ohne Prüfung → NICHT ABGENOMMEN
G9	Geprüft wird nur, was in diesem Lauf geschrieben wurde; eine Datei aus einem früheren Lauf kann kein Ergebnis liefern.	pruefe_regeln.py: programm_testen fremd.py → FEHLER „noch nicht geschrieben“
G10	Ein Werkzeugaufruf wird auch dann erkannt, wenn ein kleines Modell das Wort WERKZEUG: weglässt — der Name eines bekannten Werkzeugs am Zeilenanfang mit JSON genügt; ein Name mitten im Satz nicht.	pruefe_regeln.py, vier Fälle mit Gegenproben

	Anforderung	geprüft durch
G11	aufraeumen steht dem Modell nicht zur Verfügung: Es beendet den Modellserver und löscht die Ablage samt Protokoll. Ruft das Modell es dennoch auf, bekommt es eine Abweisung, der Lauf geht weiter. Aufgeräumt wird am Kursende über den Knopf.	pruefe_regeln.py: Aufruf abgewiesen, Ablage und Protokoll bleiben
G12	Das Modell im Paket ist ein Coder-Modell (Qwen2.5-Coder, 3B/7B), weil die Aufgabe ein Modell speziell für Python verlangt; bei gleicher Größe wird es dem Instruct-Modell vorgezogen. Instruct bleibt Rückfall.	hole_paket.py --modell passend holt Coder; modell.waehlen sortiert Coder vor; Messreihe 04.10.2026

H — Hardware

	Anforderung	geprüft durch
H1	Der ESP32 darf fabrikneu oder beliebig vorprogrammiert sein.	erase_flash + Zustandsbericht
H2	Vor dem Schreiben wird geprüft, ob dort überhaupt ein ESP32 antwortet.	esp32_firmware, Abbruch ohne Antwort
H3	Der Anschluss wird nie geraten; ports_zeigen schlägt vor, der Mensch bestätigt.	Werkzeugtext
H4	Die USB-Treiber liegen im Paket, von der Herstellerseite, mit Prüfsummen.	paket\treiber\LIESMICH.md
H5	Der Agent installiert keinen Treiber; das entscheidet ein Mensch.	kein Werkzeug dafür
H6	Ohne Hardware läuft die ganze Kette bis zur Abnahme.	Beispiel 2
H7	Vom Gerät zurücklesen: ob die LED wirklich blinkt, wird auf dem ESP32 selbst gemessen (Pegel, Wechsel, Takt) und gegen dieselbe Erwartung gehalten — der zweite, unabhängige Weg neben programm_testen.	esp32_nachlesen; 04.10.2026 an COM3: 1,04 Hz bestanden, Gegenprobe gegen 2 Hz durchgefallen, 2-Hz-Programm 2,10 Hz

	Anforderung	geprüft durch
H8	Das Rücklesen misst Flanken per Interrupt mit dem Mikrosekundenzähler des Geräts, nicht durch Abtasten.	Laptop 04.10.2026: 2,0000 s / 0,500 Hz
H9	esp32_uebertragen liest nach dem Neustart am Gerät nach, wenn eine Erwartung bekannt ist.	Werkzeugtext

Benannt und gewollt: Der ESP32 **wird** verändert — sein Speicher wird gelöscht und neu beschrieben. Das ist die einzige bleibende Änderung, die das Paket macht, und sie trifft das Steckbrett, nicht den Rechner.

I — Ferndiagnose

Im Kurs sitzt niemand daneben, der erklärt, was er sieht. Es muss eine Datei geben, die das für ihn tut.

	Anforderung	geprüft durch
I1	Ein Befehl erzeugt eine Datei, die zur Beurteilung von außen genügt: Rechner, Paket, Selbstprüfung, Werkzeugprobe, Protokoll, Ablage.	sammle_bericht.py, sechs Abschnitte
I2	Der Bericht nennt weder Benutzer- noch Rechnernamen.	Gegenprobe: Rechnernamen im Bericht suchen
I3	Fehlt etwas — ein Werkzeug, das eigene Python, das Modell — steht das im Bericht, statt den Bericht scheitern zu lassen.	Lauf ohne gefülltes Paket
I4	Die Frage „verändert das meinen Rechner?“ lässt sich selbst beantworten, ohne uns zu fragen.	pruefe_unversehrtheit.py: Durchsicht, Verbotsliste und gemessener Lauf
I5	Bricht der Agent ab, bleibt ein Fehlerbericht (ablage\fehlerbericht.txt): Auftrag, Fehler, Rückverfolgung, Modell, Signaturpflicht, die letzten 40 Protokollzeilen, was zu tun ist. Das Protokoll endet mit ABBRUCH und ENDE, nie mitten im Satz.	pruefe_regeln.py: erzwungener Abbruch → Bericht und Protokollende vorhanden

	Anforderung	geprüft durch
I6	Ist das Kontextfenster des Modells voll, endet der Lauf nicht: ältere Schritte werden zusammengefasst, der Lauf geht weiter. Werkzeugergebnisse gehen gekürzt ins Gedächtnis des Modells, vollständig ins Protokoll.	pruefe_regeln.py: KontextVoll → Zusammenfassung → FERTIG

Benannt: Ein Bericht wandert zu jemandem, der ihn liest. Er soll sagen, was das Paket tut, und nicht, wem der Rechner gehört.

J – Das Gespräch

Anweisung des Auftraggebers (04.10.2026): „Am Ende muss wie auch hier immer die Möglichkeit zur Eingabe der nächsten Anweisung bestehen ... unten immer das Eingabefeld, oben der Kopf, und das Fenster mit der Ausgabe wird gescrollt bei Bedarf ... ein Test ist dann, über die Eingabe zu sagen: verwende einen anderen Pin, oder ändere die Blinkfrequenz, oder prüfe zuerst, bevor du das Programm runterlädst, oder ich gebe selbst ein Programm ein zum Überprüfen.“

	Anforderung	geprüft durch
J1	Die Oberfläche ist ein Gespräch: Kopf oben, Verlauf in der Mitte (scrollt, folgt dem Leser), Eingabefeld immer unten. Nach jeder Anweisung ist die nächste möglich.	Browser-Prüfung (Playwright): Lage der drei Bereiche, Eingabe nach dem Ende frei, Verlauf folgt bis unten
J2	Der Agent behält den Zusammenhang : Verlauf, geschriebene und abgenommene Dateien, genannte Anschlüsse, die Erwartung; der Modellserver wird nicht je Anweisung neu geladen.	pruefstand/pruefe_gespraech.py: sechs Anweisungen, ein Modellstart, Verlauf > 20 Nachrichten
J3	Anderer Pin / andere Frequenz aus einem Satz („Nimm GPIO 4“, „10 Hz“, „5-mal pro Sekunde“): die Erwartung wird Stück für Stück fortgeführt, die betroffene Datei gilt als neu zu prüfen, FERTIG erst nach bestandenem Test gegen die neue Erwartung.	pruefe_gespraech.py Anweisungen 2, 2b, 3; Browser mit Coder-3B 1 Hz → GPIO 4 → 10 Hz
J4	Erst prüfen, dann laden: esp32_uebertragen nimmt nur eine Datei, die zuletzt geprüft und bestanden hat und seitdem unverändert ist.	pruefe_gespraech.py Anweisung 4

	Anforderung	geprüft durch
J5	Der Anschluss kommt vom Menschen: ein Anschluss, den keine Anweisung genannt hat, wird nicht verwendet; der Agent soll fragen.	pruefe_gespraech.py Anweisung 4 (COM9 abgewiesen, COM3 genannt)
J6	Eigenes Programm: steht in der Anweisung ein Codeblock (), schreibt der Agent ihn wörtlich in die Werkstatt — kein Abschreiben durch das Modell —, und das Modell prüft nur noch.	pruefe_gespraech.py Anweisung 5: Inhalt byteweise gleich
J7	Mehrere Werkzeugaufrufe in einer Antwort werden der Reihe nach ausgeführt; ein Aufruf ohne JSON gilt als leere Eingabe; ein FERTIG, das etwas behauptet, was kein Werkzeug getan hat („auf den ESP32 überspielt“), wird zurückgewiesen.	pruefe_regeln.py
J8	Agent und Modellanbindung lassen sich im Stillstand neu laden (/neu laden), die Seite wird je Aufruf gelesen — Korrekturen des Dozenten brauchen keinen Neustart.	04.10.2026: dreimal angewendet
J9	Sagt das Modell ein zweites Mal FERTIG, ohne die geänderte Datei geprüft zu haben, prüft der Agent selbst — einmal je Anweisung, sichtbar als eigener Aufruf, mit der Erwartung des Menschen (ESP32-Programm → programm_testen, sonst programm_ausfuehren). Der Prüfschritt ist nicht verhandelbar.	pruefe_ungeprueft.py; Browser 04.10.2026 15:09: „AGENT PRUEFT SELBST blink.py“ → BESTANDEN → FERTIG
J10	Quereingabe: Das Eingabefeld bleibt während des Laufs offen; eine Eingabe wird als Zwischenruf eingereicht und beim nächsten Schritt aufgenommen (Anschluss nennen, Erwartung ändern, korrigieren). Anschlüsse gelten auch als „com 3“.	pruefe_gespraech.py

	Anforderung	geprüft durch
J11	Was eine Anweisung verlangt (laden, nachlesen), muss in dieser Anweisung gelaufen sein; eine Behauptung im FERTIG ohne Werkzeug wird zurückgewiesen, dreimal: NICHT ABGENOMMEN.	pruefe_gespraech.py 5b
J12	Die Oberfläche zeigt bei länger laufenden Werkzeugen einen Fortschrittsbalken aus der erwarteten Dauer („den Download anzeigen“).	Browser
J13	Ergebnisoffen: Das Modell antwortet frei. Gruß, Frage, Bemerkung werden in Worten beantwortet, eine Aufgabe mit Werkzeugen. Der Agent steuert nicht (kein Zwang zum Werkzeugaufruf, kein Nachhaken), er wacht über die Ehrlichkeit: Erwartung des Menschen bleibt, nichts wird behauptet, kein Anschluss geraten, nur Geprüftes aufs Gerät. Anlass 05.10.2026: „hallo bist du bereit?“ wurde zu einem erfundenen Blinkauftrag; „keine ergebnisoffene Eingabe, sondern alles vorgegeben, das war nicht der Sinn der Übung“.	pruefe_gespraech.py 5c; Laptop-Gespräch
J14	Was der Mensch an der Erwartung nicht festlegt, legt der erste Aufruf des Modells fest und bleibt (sonst wandert sie von 10 Hz zu 1 Hz, je nach Programm).	pruefe_gespraech.py 5d
J15	Der Agent kennt die Werkzeuge und die Methode (schreiben, prüfen, berichtigen, ausliefern, nachlesen) und stellt sie dem Modell je nach dessen Fähigkeit bereit: als Beschreibung in der Anweisung, als Vorschlag im Ergebnis („NÄCHSTER SCHRITT“), als vorgerechnete Zahl (Takt) — und übernimmt den Prüfschritt selbst, wenn das Modell ihn auslässt. Das gehört ins Manuskript (Anweisung 05.10.2026).	Anweisungstext, Hinweise, pruefe_ungeprueft.py; Manuskript F8

	Anforderung	geprüft durch
J16	Fokus: Erzählt das Modell Arbeit, die kein Werkzeug getan hat, oder verlangt die Anweisung einen Geräteschritt und nichts lief, fragt der Agent das Modell ohne die lange Vorgeschichte nur nach dem nächsten Werkzeugaufruf (Anweisung + Aufgabe + ein Satz). Kleine Modelle folgen so, wo das Gespräch sie in Prosa hält.	pruefe_gespraech.py 5g
J17	Fehlt im Prüfaufruf die Datei, nimmt der Agent die zuletzt geschriebene dieser Anweisung und sagt es; eine Behauptung in freier Rede ohne Beleg bekommt eine sichtbare Anmerkung des Agenten.	pruefe_gespraech.py 5f; Laptop 05.10.2026 9:55
J18	Die Erwartung des Menschen wird aus seinen Worten gelesen, auch Zahlwörter („eine Periode pro Sekunde“, „zweimal je Sekunde“) und die Kurvenform („sinusförmig“). Was er nicht festlegt, legt der erste Aufruf des Modells fest — außer unsichtbare Takte (> 50 Hz, Trägerfrequenz) und mindestens_wechsel.	erwartung_aus-Proben; Laptop 05.10.2026 9:44 (1000 Hz) und 10:17 (10 Wechsel)

K — Allgemeine Python-Programme

Anweisung des Auftraggebers (04.10.2026): „oder programmiere mir mal einen einfachen Taschenrechner mit graphischer Ausgabe.“ Die Aufgabe verlangt Python-Programme auf Anforderung — nicht nur für den ESP32.

	Anforderung	geprüft durch
K1	programm_ausfuehren führt ein beliebiges Programm aus der Werkstatt mit dem Paket-Python aus: Ausgabe, Fehlerausgabe, Rückgabewert, Zeitgrenze, Text für input(). Urteil LAUF BESTANDEN / NICHT BESTANDEN.	Prüfstand: gutes, kaputtes und lesendes Programm

	Anforderung	geprüft durch
K2	Ein Programm mit Fenster (tkinter) wird gestartet und beobachtet: läuft es, zeigt es ein Fenster (Titel über die Prozessnummer)? Mit <code>offen_lassen</code> bleibt es für den Menschen offen.	Laptop 04.10.2026, 14:42: „Fensterprobe Kurs-Agent“ erkannt, LAUF BESTANDEN
K3	tkinter liegt im Paket (paket\tkinter-8.6.13-py3.12-win-amd64.zip, 3,1 MB, aus conda-forge, Lizenzen dabei); <code>umgebung_anlegen</code> entpackt es ins eigene Python und prüft <code>import tkinter</code> . Nichts davon berührt das System.	Laptop 04.10.2026: „Gegenprobe tkinter: Tk 8.6“
K4	<code>programm_ausfuehren</code> zählt als Prüfung wie <code>programm_testen</code> : ein kaputter Lauf sperrt FERTIG, ein guter gibt frei; nur in dieser Sitzung Geschriebenes wird ausgeführt.	<code>pruefe_regeln.py</code>
K5	<code>programm_testen</code> misst bei PWM die Hüllkurve des Tastgrads: Periode, Takt, Form (sinusförmig oder dreieckig, Abweichung zu beiden Bezugsformen); die Form ist Abnahmekriterium, wenn der Mensch sie nennt — „Dreieck ist kein Sinus“. PWM-Durchgänge zählen als Zustandswechsel.	<code>pruefstand/pruefe_pwm.py</code> 7/7
K6	Der Nachbau rechnet mit einer gedachten Uhr : <code>time.sleep</code> rückt sie vor statt zu warten. Grund: Windows hält 10 ms Schlaf nicht ein (bis 15,6 ms), der ESP32 schon; ein richtiges 1-Hz-Atmen lief auf dem PC mit 0,64 Hz. Zeitstempel und Zeitgrenze folgen der gedachten Uhr; ein Lauf dauert Bruchteile einer Sekunde. Rechenbibliotheken <code>math</code> , <code>random</code> , <code>struct</code> sind erlaubt.	<code>pruefe_pwm.py</code> , <code>pruefe_alles.py</code> 19/19

Was ausdrücklich nicht gilt

- Das Paket ist **kein Schutz gegen einen Angreifer**. Der Syntaxprüfer in `programm_testen` schützt gegen ein Modell, das sich vertut — nicht gegen jemanden, der ihn umgehen will. Wer das braucht, führt fremden Quelltext gar nicht aus.
- Die **Windows-Registrierung wird nicht gemessen**, sondern begründet: nichts im Paket ruft etwas auf, das dort schriebe. Begründet ist nicht gemessen.
- Es ist **keine Entwicklungsumgebung**. Es zeigt einen Vorgang an einer kleinen Aufgabe.

Prof. Dr.-Ing. Ralph Wystup M.Sc. — erstellt mit KI und Agent (Claude Code, Anthropic)